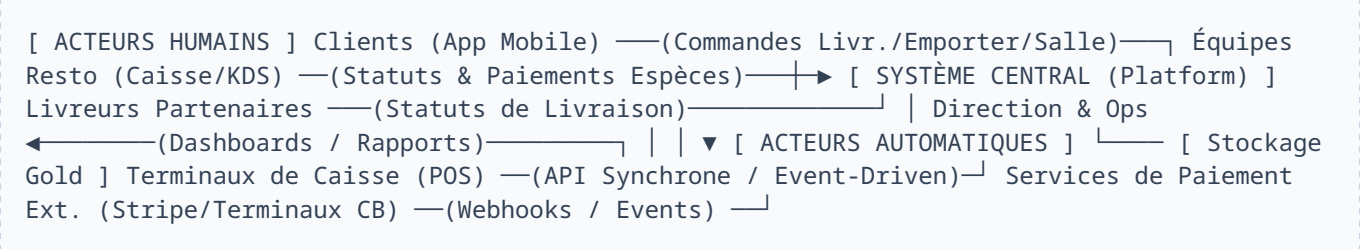
Dossier d'Architecture Technique

Plateforme de Données Temps Réel & Batch — RestaurantGroup SA

Version: 1.0.0 | **Date:** 2026-05-23 | **Statut:** Validé

Sous-partie A — Analyse du système

1. Schéma des acteurs et flux



2. Liste commentée des sources de données

Flux d'événements de commande (Événementiel, Fil de l'eau) : Émis par l'application mobile des clients, les bornes ou terminaux de caisse (POS) et les partenaires de livraison (Deliveroo, UberEats). Contient les créations et changements de statuts.

Flux de paiement (Événementiel, Fil de l'eau) : Émis par la passerelle de paiement (Stripe, Adyen) pour le web/mobile, ou par les terminaux de paiement électronique (TPE) en restaurant lors de la validation de la transaction.

Snapshots de performance (Périodique, Toutes les 30s) : Poussés automatiquement par le système local de chaque restaurant (KDS - Kitchen Display System / Logiciel de caisse) pour donner l'état de la cuisine (backlog, charge).

Référentiel Restaurants & Villes (Sur demande / Master Data) : Base de données relationnelle interne contenant la cartographie des 120 restaurants, leur ville, zone géographique et managers. Utilisé pour l'enrichissement en zone Silver.

3. Interactions synchrones vs asynchrones

ÉMETTEUR	RÉCEPTEUR	NATURE	PROTOCOLE / TECHNO	JUSTIFICATION MÉTIER
Terminal / App Mobile	order_service	Synchrone	HTTP/REST (JSON)	Garantit immédiatement au client/ caissier que la commande est prise en compte ou rejetée si invalide.
order_service	Message Broker	Asynchrone	Kafka Producer API	Le service d'ingestion publie l'événement et répond au client sans attendre le traitement complet.
Passerelle Paiement	payment_service	Asynchrone	Webhook (HTTP POST)	La banque notifie le système dès que les fonds sont validés, sans bloquer le thread de vente.
Snapshot Collector	Broker / Storage	Asynchrone	MQTT / HTTP	Envoi d'un état instantané automatisé sans attendre de réponse applicative.

Sous-partie B — Modèle de données et contrats

Les contrats formels définissent les structures strictes de nos flux de données à l'entrée de la zone Bronze.

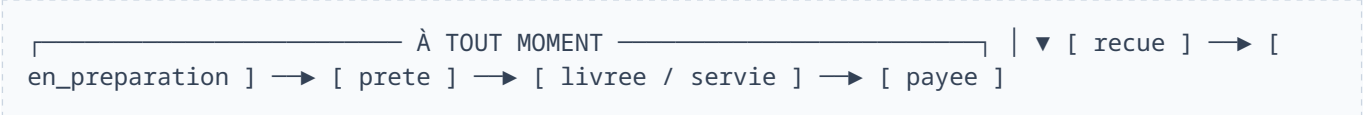
1. Règles de validation à l'ingestion

Format Métier : Rejet immédiat de toute commande avec un $amount \leq 0$ ou un $items_count < 1$.

Validation du Schéma : Rejet si un champ requis manque ou si le format de `event_ts` n'est pas une string ISO 8601 UTC valide.

Vérification Temporelle (Clock Skew) : Rejet ou marquage si `event_ts` a plus de 5 minutes d'avance sur le temps du serveur d'ingestion (sécurité anti-fraude ou bug d'horloge client).

2. Diagramme des transitions de statut d'une commande

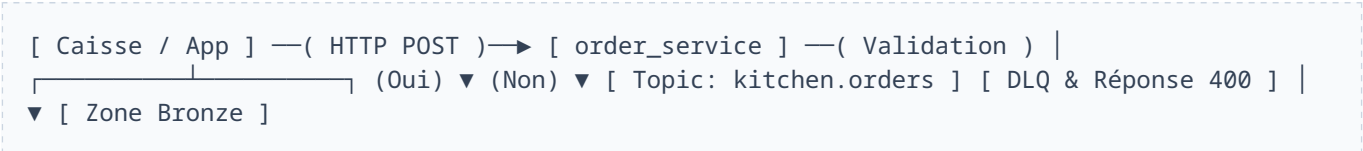


Note réglementaire : Une commande peut transiter vers le statut `annulee` depuis n'importe quel état antérieur à `payee`. Une fois payée, l'annulation se transforme réglementairement en flux de remboursement (avoir).

Sous-partie C — Ingestion et services

Le service `order_service` est une API REST synchrone ultra-légère développée pour répondre en moins de 200ms afin de ne pas bloquer les terminaux de caisse physiques.

Routage post-validation : Si la commande est valide, elle est enrichie avec un champ `server_ts` et publiée instantanément dans le topic Kafka `kitchen.orders.v1`. Si elle est invalide, elle est rejetée avec un code HTTP `400 Bad Request` et copiée dans une file de rejets (Dead Letter Queue - **DLQ**) nommée `orders.rejected.dlq` pour analyse.



Sous-partie D — Flux événementiel et partitions

1. Configuration et routage des Topics Kafka

Le choix de la clé de partitionnement est fondamental pour garantir l'ordre et l'évolutivité du système.

Pour le topic `kitchen.orders.v1`, la clé de partitionnement retenue est **`order_id`**. Cela garantit que tous les changements d'états d'une même commande atterrissent de façon déterministe dans la même partition, respectant ainsi strictement l'ordre chronologique des statuts lors de la consommation.

Si l'on avait choisi `restaurant_id` comme clé, cela créerait un déséquilibre massif de charge (un grand restaurant parisien générant 10x plus de messages qu'un petit restaurant de province sur une même partition). L'utilisation de `order_id` assure une distribution uniforme (hachage) sur toutes les partitions disponibles.

2. Mécanisme de reprise après panne & Désordre

Les instances de traitement partagent le travail via un *Consumer Group*. Chaque consumer enregistre régulièrement sa progression dans Kafka via la notion d'**offset**. En cas de panne, un rebalancing est déclenché et un autre consumer reprend la partition à partir du dernier offset stable connu.

En cas d'événements désordonnés (ex: `payee` avant `servie`), le processeur de streaming maintient un état temporaire clé/valeur local (RocksDB) par `order_id` pour réaligner l'historique de la commande grâce aux timestamps avant l'écriture ou lors de la réconciliation en zone Silver.

Sous-partie E — Stockage Bronze / Silver / Gold

L'architecture suit le paradigme **Medallion Architecture** pour isoler les étapes de traitement de données :

- **Zone Bronze :** Stockage brut, immuable et persistant des payloads JSON reçus des topics Kafka, partitionné par date de réception (`year=YYYY/month=MM/day=DD/`).
- **Zone Silver :** Données nettoyées, dédoublonnées, enrichies par le référentiel géographique (villes, régions) et standardisées (calcul du temps de préparation technique $T_{prep} = T_{prete} - T_{recue}$). Stocké au format Delta Lake.
- **Zone Gold :** Tables hautement agrégées et optimisées pour la BI et les dashboards d'exploitation (ex: chiffre d'affaires quotidien par restaurant, KPIs par ville).

Sous-partie F — Traitement Stream

1. Indicateurs de Performance Temps Réel

INDICATEUR	SOURCE (TOPIC)	LOGIQUE DE CALCUL	GRANULARITÉ	FRÉQUENCE
Commandes en cours	kitchen.orders.v1	Compteur incrémenté si 'recue'/'en_prep', décrémenté si 'prete'/'annulee'.	Par Restaurant	Fil de l'eau
Temps de prep. moyen glissant	kitchen.orders.v1	Moyenne du delta entre 'prete' et 'recue' sur une fenêtre glissante de 1h.	Par Restaurant	Toutes les 30s
Chiffre d'affaires instantané	payment.events.v1	Somme cumulée des 'amount' depuis le début de la journée.	Par Ville	Toutes les 10s

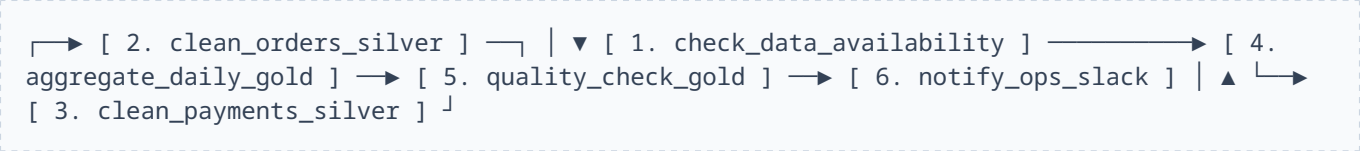
2. Gestion des données tardives (Watermarking)

Le processeur de stream applique un **Watermark de 15 minutes**. Tout événement arrivant avec moins de 15 minutes de retard par rapport à l'heure courante (Event Time vs Processing Time) est intégré rétroactivement dans les fenêtres glissantes. Au-delà, l'événement est persisté en zone Bronze pour la conformité historique mais exclu des calculs d'alertes en temps réel.

Sous-partie G — Traitement Batch & Orchestration

Les calculs financiers lourds et consolidés sont exécutés en mode batch à 2h00 du matin pour des raisons d'efficacité économique et de cohérence comptable. Le pipeline est strictement **idempotent** : l'utilisation d'opérations `UPSERT` ou `INSERT OVERWRITE` en Delta Lake garantit qu'une double exécution accidentelle d'un batch n'altère pas ou ne double pas les indicateurs financiers.

Structure du DAG d'Orchestration (nightly_dag.py)



Sous-partie I — Observabilité & Métriques

NOM DE LA MÉTRIQUE	COMPOSANT	TYPE	SEUIL D'ALERTE	DESTINATAIRE
stream_consumer_lag	Stream	Gauge	> 5000 messages	Data Ops (Astreinte)
batch_rejected_lines_count	Batch	Counter	> 1% du volume	Data Quality Analyst
gold_table_freshness_hours	Stockage	Gauge	> 25 heures	Business Intelligence